Madhura Vilas Suroshe

24102C2002

```
# ============================================================
# LAB 3: MULTI-SOURCE RETAIL SALES DATA INTEGRATION & ANALYSIS
# Platform: R / Google Colab
# ============================================================

# Install required packages
install.packages(c(
  "tidyverse",
  "jsonlite",
  "readxl",
  "writexl",
  "DBI",
  "RSQLite"
), repos = "https://cloud.r-project.org")

# Load libraries
library(tidyverse)
library(jsonlite)
library(readxl)
library(writexl)
library(DBI)
library(RSQLite)

# Download UCI Online Retail Dataset
url <- "https://archive.ics.uci.edu/static/public/352/online+retail.zip"
zip_file <- "online_retail.zip"

download.file(url, zip_file, mode = "wb")

# Extract dataset
unzip(zip_file)

# Check extracted files
list.files()

# The UCI dataset is normally provided as Online Retail.xlsx
raw_data <- read_excel("Online Retail.xlsx")

cat("Original dataset dimensions:\n")
cat("Rows:", nrow(raw_data), "\n")
cat("Columns:", ncol(raw_data), "\n")

head(raw_data)
```

```
Installing packages into ‘/usr/local/lib/R/site-library’
(as ‘lib’ is unspecified)

── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.2.1     ✓ readr     2.2.0
✓ forcats   1.0.1     ✓ stringr   1.6.0
✓ ggplot2   4.0.3     ✓ tibble    3.3.1
✓ lubridate 1.9.5     ✓ tidyr     1.3.2
✓ purrr     1.2.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

Attaching package: ‘jsonlite’


The following object is masked from ‘package:purrr’:

    flatten


'Online Retail.xlsx' · 'online_retail.zip' · 'sample_data'
Original dataset dimensions:
Rows: 541909
Columns: 8
```

A tibble: 6 × 8

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
<chr>	<chr>	<chr>	<dbl>	<dttm>	<dbl>	<dbl>	<chr>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom

```
# ============================================================
# TASK 1: IMPORT AND CLEAN THE DATA
# ============================================================

# ------------------------------------------------------------
# STEP 1: Prepare the three required source files
# ------------------------------------------------------------

# Rename columns to convenient names
retail_raw <- raw_data %>%
  rename(
    InvoiceNo = InvoiceNo,
    StockCode = StockCode,
    Description = Description,
    Quantity = Quantity,
    InvoiceDate = InvoiceDate,
    UnitPrice = UnitPrice,
    CustomerID = CustomerID,
    Country = Country
  )

# Create transactions.csv
transactions <- retail_raw %>%
  select(
    InvoiceNo,
    StockCode,
    CustomerID,
    Quantity,
    InvoiceDate
  )

write_csv(transactions, "transactions.csv")

# Create products.json
products <- retail_raw %>%
  select(StockCode, Description, UnitPrice) %>%
  filter(!is.na(StockCode),
         !is.na(Description),
         !is.na(UnitPrice),
         UnitPrice > 0) %>%
  group_by(StockCode, Description) %>%
  summarise(
```

```
    UnitPrice = median(UnitPrice),
    .groups = "drop"
  )

write_json(products, "products.json", pretty = TRUE)

# Create customers.xlsx
customers <- retail_raw %>%
  select(CustomerID, Country) %>%
  filter(!is.na(CustomerID),
         !is.na(Country)) %>%
  distinct()

write_xlsx(customers, "customers.xlsx")


# ------------------------------------------------------------
# STEP 2: IMPORT CSV, JSON AND EXCEL
# ------------------------------------------------------------

transactions_imported <- read_csv(
  "transactions.csv",
  show_col_types = FALSE
)

products_imported <- fromJSON("products.json")

customers_imported <- read_excel("customers.xlsx")


# ------------------------------------------------------------
# STEP 3: INSPECT DATA
# ------------------------------------------------------------

cat("\n================ DATA DIMENSIONS ================\n")

cat("\nTransactions:\n")
print(dim(transactions_imported))

cat("\nProducts:\n")
print(dim(products_imported))

cat("\nCustomers:\n")
print(dim(customers_imported))

cat("\nTransactions structure:\n")
str(transactions_imported)

cat("\nProducts structure:\n")
str(products_imported)

cat("\nCustomers structure:\n")
str(customers_imported)


# ------------------------------------------------------------
# STEP 4: MISSING VALUE CHECK
# ------------------------------------------------------------

cat("\n================ MISSING VALUES ================\n")

cat("\nTransactions missing values:\n")
print(colSums(is.na(transactions_imported)))

cat("\nProducts missing values:\n")
print(colSums(is.na(products_imported)))

cat("\nCustomers missing values:\n")
print(colSums(is.na(customers_imported)))


# ------------------------------------------------------------
# STEP 5: CLEAN TRANSACTIONS
# ------------------------------------------------------------

transactions_clean <- transactions_imported %>%
  filter(
    !is.na(InvoiceNo),
    !is.na(StockCode),
    !is.na(CustomerID),
    !is.na(Quantity),
    !is.na(InvoiceDate)
```

```
  ) %>%
  filter(Quantity > 0) %>%
  distinct()


# ------------------------------------------------------------
# STEP 6: CLEAN PRODUCTS
# ------------------------------------------------------------

products_clean <- products_imported %>%
  filter(
    !is.na(StockCode),
    !is.na(Description),
    !is.na(UnitPrice),
    UnitPrice > 0
  ) %>%
  distinct()


# ------------------------------------------------------------
# STEP 7: CLEAN CUSTOMERS
# ------------------------------------------------------------

customers_clean <- customers_imported %>%
  filter(
    !is.na(CustomerID),
    !is.na(Country)
  ) %>%
  distinct()


# ------------------------------------------------------------
# STEP 8: REMOVE DUPLICATE PRODUCT STOCK CODES
# ------------------------------------------------------------

products_clean <- products_clean %>%
  group_by(StockCode) %>%
  summarise(
    Description = first(Description),
    UnitPrice = median(UnitPrice),
    .groups = "drop"
  )


# ------------------------------------------------------------
# STEP 9: ADD UNIT PRICE TO TRANSACTIONS
# ------------------------------------------------------------

transactions_clean <- transactions_clean %>%
  left_join(
    products_clean %>% select(StockCode, UnitPrice),
    by = "StockCode"
  )


# ------------------------------------------------------------
# STEP 10: REMOVE TRANSACTIONS WITH INVALID PRICES
# ------------------------------------------------------------

transactions_clean <- transactions_clean %>%
  filter(
    !is.na(UnitPrice),
    UnitPrice > 0
  )


# ------------------------------------------------------------
# STEP 11: CREATE REVENUE
# ------------------------------------------------------------

transactions_clean <- transactions_clean %>%
  mutate(
    Revenue = Quantity * UnitPrice
  )


# ------------------------------------------------------------
# STEP 12: FINAL CLEANING SUMMARY
# ------------------------------------------------------------

cat("\n================ CLEANING SUMMARY ================\n")
```

```
cat("\nOriginal rows:", nrow(transactions_imported))
cat("\nClean transaction rows:", nrow(transactions_clean))
cat("\nRemoved rows:", nrow(transactions_imported) - nrow(transactions_clean))

cat("\n\nDuplicate transaction records removed:",
    nrow(transactions_imported) - nrow(distinct(transactions_imported)))

cat("\n\nInvalid/zero quantities removed:",
    sum(transactions_imported$Quantity <= 0, na.rm = TRUE))

cat("\n\nInvalid/zero prices in product data:",
    sum(products_imported$UnitPrice <= 0, na.rm = TRUE))

cat("\n\nFinal Revenue column created successfully.\n")

cat("\nCleaning decisions:\n")
cat("1. Records with missing essential transaction fields were removed.\n")
cat("2. Duplicate transaction records were removed.\n")
cat("3. Transactions with zero or negative quantities were removed.\n")
cat("4. Products with missing or non-positive prices were removed.\n")
cat("5. Customers without CustomerID or Country were excluded.\n")
cat("6. Revenue was calculated as Quantity × UnitPrice.\n")

cat("\nFinal cleaned transaction sample:\n")
print(head(transactions_clean))
```

```
Products structure:
'data.frame':   4174 obs. of  3 variables:
 $ StockCode  : chr  "10002" "10080" "10120" "10123C" ...
 $ Description: chr  "INFLATABLE POLITICAL GLOBE" "GROOVY CACTUS INFLATABLE" "DOGGY RUBBER" "HEARTS WRAPPING TAPE" ...
 $ UnitPrice  : num  0.85 0.39 0.21 0.65 0.42 0.42 0.85 0.79 1.25 1.69 ...

Customers structure:
tibble [4,380 × 2] (S3: tbl_df/tbl/data.frame)
 $ CustomerID: num [1:4380] 17850 13047 12583 13748 15100 ...
 $ Country   : chr [1:4380] "United Kingdom" "United Kingdom" "France" "United Kingdom" ...

================ MISSING VALUES ================

Transactions missing values:
  InvoiceNo   StockCode  CustomerID    Quantity InvoiceDate
          0           0      135080           0           0

Products missing values:
  StockCode Description   UnitPrice
          0           0           0

Customers missing values:
CustomerID    Country
         0          0

================ CLEANING SUMMARY ================

Original rows: 541909
Clean transaction rows: 392708
Removed rows: 149201

Duplicate transaction records removed: 5429

Invalid/zero quantities removed: 10624

Invalid/zero prices in product data: 0

Final Revenue column created successfully.

Cleaning decisions:
1. Records with missing essential transaction fields were removed.
2. Duplicate transaction records were removed.
3. Transactions with zero or negative quantities were removed.
4. Products with missing or non-positive prices were removed.
5. Customers without CustomerID or Country were excluded.
6. Revenue was calculated as Quantity × UnitPrice.

Final cleaned transaction sample:
# A tibble: 6 × 7
  InvoiceNo StockCode CustomerID Quantity InvoiceDate         UnitPrice Revenue
  <chr>     <chr>          <dbl>    <dbl> <dttm>                  <dbl>   <dbl>
1 536365    85123A         17850        6 2010-12-01 08:26:00      2.95    17.7
2 536365    71053          17850        6 2010-12-01 08:26:00      6.02    36.1
3 536365    84406B         17850        8 2010-12-01 08:26:00      4.15    33.2
4 536365    84029G         17850        6 2010-12-01 08:26:00      4.25    25.5
5 536365    84029E         17850        6 2010-12-01 08:26:00      4.25    25.5
6 536365    22752          17850        2 2010-12-01 08:26:00      8.5     17
```

```
# ============================================================
# TASK 2: INTEGRATE MULTIPLE DATA SOURCES
```

```
# ============================================================

# ------------------------------------------------------------
# STEP 1: JOIN TRANSACTIONS WITH PRODUCT INFORMATION
# ------------------------------------------------------------

sales_products <- transactions_clean %>%
  left_join(
    products_clean,
    by = "StockCode",
    suffix = c("_transaction", "_product")
  )


# ------------------------------------------------------------
# STEP 2: JOIN WITH CUSTOMER INFORMATION
# ------------------------------------------------------------

final_retail_data <- sales_products %>%
  left_join(
    customers_clean,
    by = "CustomerID"
  )


# ------------------------------------------------------------
# STEP 3: CLEAN COLUMN NAMES
# ------------------------------------------------------------

final_retail_data <- final_retail_data %>%
  select(
    InvoiceNo,
    StockCode,
    CustomerID,
    Quantity,
    InvoiceDate,
    Description,
    UnitPrice = UnitPrice_transaction,
    Country,
    Revenue
  )


# ------------------------------------------------------------
# STEP 4: VERIFY FINAL DIMENSIONS
# ------------------------------------------------------------

cat("\n================ FINAL DATASET DIMENSIONS ================\n")

cat("Rows:", nrow(final_retail_data), "\n")
cat("Columns:", ncol(final_retail_data), "\n")

cat("\nColumn names:\n")
print(names(final_retail_data))


# ------------------------------------------------------------
# STEP 5: IDENTIFY UNMATCHED PRODUCT RECORDS
# ------------------------------------------------------------

unmatched_products <- transactions_clean %>%
  anti_join(
    products_clean,
    by = "StockCode"
  )

cat("\n================ UNMATCHED PRODUCTS ================\n")
cat("Number of unmatched product records:",
    nrow(unmatched_products), "\n")


# ------------------------------------------------------------
# STEP 6: IDENTIFY UNMATCHED CUSTOMER RECORDS
# ------------------------------------------------------------

unmatched_customers <- transactions_clean %>%
  anti_join(
    customers_clean,
    by = "CustomerID"
  )

cat("\n================ UNMATCHED CUSTOMERS ================\n")
```

```
cat("Number of unmatched customer records:",
    nrow(unmatched_customers), "\n")


# ------------------------------------------------------------
# STEP 7: VERIFY MISSING VALUES AFTER INTEGRATION
# ------------------------------------------------------------

cat("\n================ INTEGRATED DATA CHECK ================\n")

print(colSums(is.na(final_retail_data)))


# ------------------------------------------------------------
# STEP 8: DISPLAY FINAL DATASET
# ------------------------------------------------------------

cat("\n================ FINAL INTEGRATED DATA ================\n")

print(head(final_retail_data, 10))


# ------------------------------------------------------------
# STEP 9: JOIN JUSTIFICATION
# ------------------------------------------------------------

cat("\n================ JOIN JUSTIFICATION ================\n")

cat(
  "A LEFT JOIN was selected because all valid transaction records\n",
  "must be retained while product and customer information are added.\n",
  "This prevents valid sales transactions from being lost during integration.\n"
)
```

```
Warning message in left_join(., customers_clean, by = "CustomerID"):
“Detected an unexpected many-to-many relationship between `x` and `y`.
ℹ Row 196 of `x` matches multiple rows in `y`.
ℹ Row 1 of `y` matches multiple rows in `x`.
ℹ If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.”

================ FINAL DATASET DIMENSIONS ================
Rows: 393617
Columns: 9

Column names:
[1] "InvoiceNo"   "StockCode"   "CustomerID"  "Quantity"    "InvoiceDate"
[6] "Description" "UnitPrice"   "Country"     "Revenue"

================ UNMATCHED PRODUCTS ================
Number of unmatched product records: 0

================ UNMATCHED CUSTOMERS ================
Number of unmatched customer records: 0

================ INTEGRATED DATA CHECK ================
  InvoiceNo   StockCode  CustomerID    Quantity InvoiceDate Description
          0           0           0           0           0           0
  UnitPrice     Country     Revenue
          0           0           0

================ FINAL INTEGRATED DATA ================
# A tibble: 10 × 9
   InvoiceNo StockCode CustomerID Quantity InvoiceDate         Description
   <chr>     <chr>     <dbl>       <dbl> <dttm>              <chr>
 1 536365    85123A    17850           6 2010-12-01 08:26:00 CREAM HANGING HE…
 2 536365    71053     17850           6 2010-12-01 08:26:00 WHITE METAL LANT…
 3 536365    84406B    17850           8 2010-12-01 08:26:00 CREAM CUPID HEAR…
 4 536365    84029G    17850           6 2010-12-01 08:26:00 KNITTED UNION FL…
 5 536365    84029E    17850           6 2010-12-01 08:26:00 RED WOOLLY HOTTI…
 6 536365    22752     17850           2 2010-12-01 08:26:00 SET 7 BABUSHKA N…
 7 536365    21730     17850           6 2010-12-01 08:26:00 GLASS STAR FROST…
 8 536366    22633     17850           6 2010-12-01 08:28:00 HAND WARMER UNIO…
 9 536366    22632     17850           6 2010-12-01 08:28:00 HAND WARMER RED …
10 536367    84879     13047          32 2010-12-01 08:34:00 ASSORTED COLOUR …
# ℹ 3 more variables: UnitPrice <dbl>, Country <chr>, Revenue <dbl>

================ JOIN JUSTIFICATION ================
A LEFT JOIN was selected because all valid transaction records
 must be retained while product and customer information are added.
 This prevents valid sales transactions from being lost during integration.
```

```
# Fix duplicate CustomerID mappings in customer data
customers_clean <- customers_clean %>%
  group_by(CustomerID) %>%
  summarise(
```

```
    Country = first(Country),
    .groups = "drop"
  )

# Recreate the integrated dataset
final_retail_data <- transactions_clean %>%
  left_join(
    products_clean,
    by = "StockCode",
    suffix = c("_transaction", "_product")
  ) %>%
  left_join(
    customers_clean,
    by = "CustomerID"
  ) %>%
  select(
    InvoiceNo,
    StockCode,
    CustomerID,
    Quantity,
    InvoiceDate,
    Description,
    UnitPrice = UnitPrice_transaction,
    Country,
    Revenue
  )

cat("Corrected integrated dataset:\n")
cat("Rows:", nrow(final_retail_data), "\n")
cat("Columns:", ncol(final_retail_data), "\n")

cat("\nMissing values:\n")
print(colSums(is.na(final_retail_data)))
```

```
Corrected integrated dataset:
Rows: 392708
Columns: 9

Missing values:
  InvoiceNo   StockCode  CustomerID    Quantity InvoiceDate Description
          0           0           0           0           0           0
  UnitPrice     Country     Revenue
          0           0           0
```

```
# ============================================================
# TASK 3: SALES AND CUSTOMER ANALYSIS
# ============================================================

# ------------------------------------------------------------
# 1. TOTAL SALES REVENUE
# ------------------------------------------------------------

total_revenue <- final_retail_data %>%
  summarise(
    Total_Revenue = sum(Revenue, na.rm = TRUE)
  )

cat("\n================ TOTAL SALES REVENUE ================\n")
print(total_revenue)


# ------------------------------------------------------------
# 2. TOP 5 PRODUCTS BASED ON REVENUE
# ------------------------------------------------------------

top_5_products <- final_retail_data %>%
  group_by(StockCode, Description) %>%
  summarise(
    Total_Revenue = sum(Revenue, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue)) %>%
  slice_head(n = 5)

cat("\n================ TOP 5 PRODUCTS ================\n")
print(top_5_products)


# ------------------------------------------------------------
# 3. TOP 5 COUNTRIES BASED ON REVENUE
# ------------------------------------------------------------
```

```
top_5_countries <- final_retail_data %>%
  group_by(Country) %>%
  summarise(
    Total_Revenue = sum(Revenue, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue)) %>%
  slice_head(n = 5)

cat("\n================ TOP 5 COUNTRIES ================\n")
print(top_5_countries)


# ------------------------------------------------------------
# 4. TOP 5 CUSTOMERS BASED ON TOTAL PURCHASE VALUE
# ------------------------------------------------------------

top_5_customers <- final_retail_data %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Purchase_Value = sum(Revenue, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Purchase_Value)) %>%
  slice_head(n = 5)

cat("\n================ TOP 5 CUSTOMERS ================\n")
print(top_5_customers)


# ------------------------------------------------------------
# 5. CUSTOMER VALUE CLASSIFICATION USING case_when()
# ------------------------------------------------------------

customer_value <- final_retail_data %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Purchase_Value = sum(Revenue, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    Customer_Category = case_when(
      Total_Purchase_Value < 500 ~ "Low Value",
      Total_Purchase_Value < 2000 ~ "Medium Value",
      Total_Purchase_Value < 10000 ~ "High Value",
      TRUE ~ "Premium"
    )
  )


cat("\n================ CUSTOMER VALUE CLASSIFICATION ================\n")
print(head(customer_value, 10))


# ------------------------------------------------------------
# 6. NUMBER OF CUSTOMERS IN EACH CATEGORY
# ------------------------------------------------------------

customer_category_summary <- customer_value %>%
  count(Customer_Category) %>%
  arrange(desc(n))

cat("\n================ CUSTOMER CATEGORY SUMMARY ================\n")
print(customer_category_summary)


# ------------------------------------------------------------
# 7. REVENUE BY MARKET
# ------------------------------------------------------------

market_performance <- final_retail_data %>%
  group_by(Country) %>%
  summarise(
    Total_Revenue = sum(Revenue, na.rm = TRUE),
    Total_Transactions = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue))


# ------------------------------------------------------------
# 8. HIGH-PERFORMING MARKET
```

```
# ------------------------------------------------------------

high_performing_market <- market_performance %>%
  slice_max(
    order_by = Total_Revenue,
    n = 1
  )

cat("\n================ HIGH-PERFORMING MARKET ================\n")
print(high_performing_market)


# ------------------------------------------------------------
# 9. UNDERPERFORMING MARKET
# ------------------------------------------------------------

underperforming_market <- market_performance %>%
  slice_min(
    order_by = Total_Revenue,
    n = 1
  )

cat("\n================ UNDERPERFORMING MARKET ================\n")
print(underperforming_market)


# ------------------------------------------------------------
# 10. INTERPRETATION
# ------------------------------------------------------------

cat("\n================ MARKET INTERPRETATION ================\n")

cat(
  "\nThe high-performing market is",
  high_performing_market$Country,
  "with revenue of",
  round(high_performing_market$Total_Revenue, 2),
  ".\n"
)

cat(
  "It can be considered high-performing because it generated the highest\n",
  "revenue among all countries in the dataset.\n"
)

cat(
  "\nThe underperforming market is",
  underperforming_market$Country,
  "with revenue of",
  round(underperforming_market$Total_Revenue, 2),
  ".\n"
)

cat(
  "It is considered underperforming because it generated the lowest\n",
  "revenue among the countries represented in the dataset.\n"
)


# ------------------------------------------------------------
# 11. DISPLAY COMPLETE CUSTOMER ANALYSIS
# ------------------------------------------------------------

cat("\n================ CUSTOMER ANALYSIS COMPLETE ================\n")

print(customer_value)
```

```
2 Medium Value       1668
3 High Value          849
4 Premium             126

================ HIGH-PERFORMING MARKET ================
# A tibble: 1 × 3
  Country        Total_Revenue Total_Transactions
  <chr>                  <dbl>              <int>
1 United Kingdom      8429657.             349212

================ UNDERPERFORMING MARKET ================
# A tibble: 1 × 3
  Country      Total_Revenue Total_Transactions
  <chr>                <dbl>              <int>
1 Saudi Arabia          163.                  9

================ MARKET INTERPRETATION ================

The high-performing market is United Kingdom with revenue of 8429657 .
It can be considered high-performing because it generated the highest
 revenue among all countries in the dataset.

The underperforming market is Saudi Arabia with revenue of 162.96 .
It is considered underperforming because it generated the lowest
 revenue among the countries represented in the dataset.

================ CUSTOMER ANALYSIS COMPLETE ================
# A tibble: 4,339 × 3
   CustomerID Total_Purchase_Value Customer_Category
        <dbl>                <dbl> <chr>
 1      12346               92769. Premium
 2      12347                4601. High Value
 3      12348                1690. Medium Value
 4      12349                1507. Medium Value
 5      12350                 315. Low Value
 6      12352                1512. Medium Value
 7      12353                  89  Low Value
 8      12354                1053. Medium Value
 9      12355                 459. Low Value
10      12356                3002. High Value
# ℹ 4,329 more rows
```

```
# ============================================================
# TASK 4: SQLITE DATABASE AND SQL QUERIES
# ============================================================

# ------------------------------------------------------------
# STEP 1: CREATE SQLITE DATABASE
# ------------------------------------------------------------

db <- dbConnect(
  SQLite(),
  "retail_sales.db"
)


# ------------------------------------------------------------
# STEP 2: EXPORT FINAL DATASET TO retail_sales TABLE
# ------------------------------------------------------------

dbWriteTable(
  db,
  "retail_sales",
  final_retail_data,
  overwrite = TRUE
)

cat("\n================ DATABASE CREATED ================\n")
cat("Database: retail_sales.db\n")
cat("Table: retail_sales\n")


# ------------------------------------------------------------
# STEP 3: VERIFY TABLE
# ------------------------------------------------------------

cat("\n================ DATABASE TABLES ================\n")

print(dbListTables(db))


# ------------------------------------------------------------
# STEP 4: CHECK TABLE STRUCTURE
# ------------------------------------------------------------

cat("\n================ TABLE STRUCTURE ================\n")
```

```
print(dbGetQuery(
  db,
  "PRAGMA table_info(retail_sales)"
))


# ------------------------------------------------------------
# SQL QUERY 1: TOP 5 CUSTOMERS
# ------------------------------------------------------------

query1 <- "
SELECT
    CustomerID,
    ROUND(SUM(Revenue), 2) AS Total_Revenue
FROM retail_sales
GROUP BY CustomerID
ORDER BY Total_Revenue DESC
LIMIT 5
"

top_customers_sql <- dbGetQuery(db, query1)

cat("\n================ SQL QUERY 1 ================\n")
cat("Top 5 customers based on revenue:\n")
print(top_customers_sql)


# ------------------------------------------------------------
# SQL QUERY 2: TOTAL REVENUE BY COUNTRY
# ------------------------------------------------------------

query2 <- "
SELECT
    Country,
    ROUND(SUM(Revenue), 2) AS Total_Revenue
FROM retail_sales
GROUP BY Country
ORDER BY Total_Revenue DESC
"

country_revenue_sql <- dbGetQuery(db, query2)

cat("\n================ SQL QUERY 2 ================\n")
cat("Total revenue by country:\n")
print(head(country_revenue_sql, 10))


# ------------------------------------------------------------
# STEP 5: THREE BUSINESS INSIGHTS
# ------------------------------------------------------------

cat("\n================ BUSINESS INSIGHTS ================\n")

top_country <- country_revenue_sql %>%
  slice_max(Total_Revenue, n = 1)

top_customer <- top_customers_sql %>%
  slice_max(Total_Revenue, n = 1)

top_product <- top_5_products %>%
  slice_max(Total_Revenue, n = 1)


cat(
  "\nInsight 1 - Market:\n",
  top_country$Country,
  "is the highest revenue-generating country with revenue of",
  round(top_country$Total_Revenue, 2),
  ".\n"
)

cat(
  "\nInsight 2 - Customer:\n",
  "Customer",
  top_customer$CustomerID,
  "has the highest purchase value among customers in the dataset,\n",
  "with total revenue of",
  round(top_customer$Total_Revenue, 2),
  ".\n"
)

cat(
```

```
  "\nInsight 3 - Product:\n",
  top_product$Description,
  "is the highest revenue-generating product in the analysis,\n",
  "with revenue of",
  round(top_product$Total_Revenue, 2),
  ".\n"
)


# ------------------------------------------------------------
# STEP 6: VERIFY DATA STORED IN SQLITE
# ------------------------------------------------------------

cat("\n================ SQLITE RECORD COUNT ================\n")

print(
  dbGetQuery(
    db,
    "SELECT COUNT(*) AS Total_Rows FROM retail_sales"
  )
)


# ------------------------------------------------------------
# STEP 7: CLOSE DATABASE CONNECTION
# ------------------------------------------------------------

dbDisconnect(db)

cat("\nSQLite database connection closed successfully.\n")
cat("retail_sales.db has been created successfully.\n")
```

```
================ DATABASE CREATED ================
Database: retail_sales.db
Table: retail_sales

================ DATABASE TABLES ================
[1] "retail_sales"

================ TABLE STRUCTURE ================
  cid        name type notnull dflt_value pk
1   0   InvoiceNo TEXT       0         NA  0
2   1   StockCode TEXT       0         NA  0
3   2  CustomerID REAL       0         NA  0
4   3    Quantity REAL       0         NA  0
5   4 InvoiceDate REAL       0         NA  0
6   5 Description TEXT       0         NA  0
7   6   UnitPrice REAL       0         NA  0
8   7     Country TEXT       0         NA  0
9   8     Revenue REAL       0         NA  0

================ SQL QUERY 1 ================
Top 5 customers based on revenue:
  CustomerID Total_Revenue
1      18102      403024.2
2      14646      329710.5
3      17450      181056.5
4      14156      170378.0
5      16446      168472.5

================ SQL QUERY 2 ================
Total revenue by country:
          Country Total_Revenue
1  United Kingdom    8429656.92
2     Netherlands     335170.86
3            EIRE     327189.04
4         Germany     238814.81
5          France     208476.55
6       Australia     160921.36
7           Spain      62671.58
8     Switzerland      57334.31
9         Belgium      43016.22
```